

Agent Harness Research System v4

Architecture Retrospective (A → J5.0a)

Executive Summary

The project began as a focused research assistant for NVIDIA AI infrastructure.

Over time it evolved into a general-purpose research platform capable of:

- Multi-source retrieval
- Evidence extraction
- Contradiction detection
- Research gap analysis
- Evaluation and regression testing
- Durable research state management
- Domain-specific configuration through profiles
- Functional-agent orchestration

The architecture has shifted from:

```
Question
↓
Prompt
↓
Answer
```

to:

```
Research Object
↓
Profiles
↓
Functional Agents
↓
Evidence
↓
Findings
↓
Outputs
```

Phase A – Initial Research Harness

Goal

Create a local-document research workflow for AI infrastructure topics.

Initial capabilities:

```
PDF ingestion
Markdown ingestion
Chunking
Basic retrieval
Memo generation
```

Architecture:

```
Question
↓
Sources
↓
Answer
```

Phase B – Evidence-Oriented Research

Goal

Move from answer generation to evidence-based reasoning.

Added:

```
Evidence Extraction
Evidence Ranking
Evidence IDs
Citation Tracking
```

Architecture:

Question
↓
Evidence
↓
Answer

The system became auditable.

Phase C – Contradiction Detection

Goal

Detect conflicting claims across evidence.

Added:

Entity Extraction
Metric Extraction
Scope Detection
Contradiction Identification

Examples:

120 kW
vs
200 kW

for the same rack.

Phase D – Coverage Analysis

Goal

Identify missing evidence.

Added:

Coverage Matrix
Research Gaps
Topic Coverage

The system began tracking:

What we know
What we do not know

instead of only producing answers.

Phase E – Evaluation Framework

Goal

Measure quality objectively.

Added:

Gold Questions
Gold Contradictions
Fact Coverage Scoring
Citation Coverage Scoring
Hallucination Detection

This introduced:

Benchmarking

to the platform.

Phase F – Regression Protection

Goal

Prevent improvements from breaking existing behavior.

Added:

Baseline Reports
Regression Reports
Automated Pass/Fail Gates

Architecture:

Code Change
↓
Benchmark
↓
Regression Check
↓
Accept / Reject

Phase G – Web Retrieval (K1)

Goal

Extend research beyond local documents.

Added:

Web Search
Web Download
Web Caching
Web Evidence

Architecture:

Local Sources
+
Web Sources
↓
Unified Evidence Set

Phase H – Retrieval Planning (J3.0)

Goal

Improve retrieval quality.

Added:

Retrieval Planner
Query Expansion
Planner Modes
Entity Locking
Metric Locking

Discovery:

More Retrieval
≠
Better Retrieval

Phase I – Evidence Quality (J3.1)

Goal

Improve extracted facts.

Added:

Evidence Typing
Topic Classification
Multi-Claim Extraction
Evidence Density Metrics

Result:

Substantial benchmark improvement.

Phase J – Semantic Benchmarking

Goal

Reduce benchmark brittleness.

Added:

Semantic Matching
Synonym Registry
Anti-Synonym Registry
Benchmark Hygiene

Discovery:

Evaluation quality
was a major bottleneck.

Many apparent failures were benchmark artifacts.

Phase K – Research Objects (J4.5)

Goal

Create durable research state.

Before:

Question
↓
Answer

After:

Research Object
↓
Evidence
↓
Findings

↓
Answer

Research Objects store:

```
{  
  "question": "...",  
  "profile": "...",  
  "evidence": [...],  
  "findings": [...],  
  "research_gaps": [...]  
}
```

This became the central system artifact.

Phase L – Profile Architecture

Goal

Remove domain knowledge from code.

Before:

```
Python  
contains  
domain knowledge
```

After:

```
Profiles  
contain  
domain knowledge
```

Added:

```
profiles/ai_data_centers.yaml  
profiles/smr.yaml
```

Domain concepts moved out of:


```
agent.py
coverage.py
gap_detector.py
retrieval_planner.py
source_quality.py
```

into profile definitions.

Result:

```
New domain
=
New YAML
```

No Python changes required.

Phase M – Profile Editor

Goal

Allow domain experts to modify behavior.

Added:

```
FastAPI Backend
React Frontend
Profile CRUD API
Profile Editing UI
```

Architecture:

```
Domain Expert
↓
Profile Editor
↓
YAML Profile
↓
Research Platform
```

This separated:

Software Development

from:

Knowledge Management

Phase N – Benchmark Stabilization

Goal

Restore benchmark stability after Research Objects.

Added:

Profile Validation
Benchmark Domain Mapping
Research Object Validation

Final state:

Overall Score	0.9928
Fact Coverage	0.9783
Hallucination Rate	0.0000
Citation Coverage	1.0000
Contradiction Acc.	1.0000
Q&A Passed	23 / 23
Regression	PASS

Phase O – Functional Agent Architecture (J5.0a)

Major Architectural Shift

Originally the roadmap considered:

Domain Agents

Examples:

Nuclear Agent
Data Center Agent
Cooling Agent
Networking Agent

This approach was rejected.

Reason:

Research questions routinely span multiple domains.

Example:

Can SMRs power AI data centers?

Crosses:

SMR
Data Centers
Power
Economics
Regulation

Domain-agent ownership becomes ambiguous.

New Principle

Separate:

Knowledge

from:

Behavior

Profiles = Knowledge

Profiles answer:

What domain knowledge exists?

Examples:

SMR
AI Data Centers
Transmission
Fusion

Profiles contain:

Topics
Entities
Perspectives
Source Rules
Retrieval Rules

Functional Agents = Behavior

Agents answer:

What should happen?

Examples:

Planner Agent
Evidence Agent
QA Agent
Report Agent

Agents are domain-independent.

Functional Agent Architecture

Target structure:

```
Research Object
+
Profiles
+
Functional Agents
```

Workflow:

```
Question
↓
Research Object
↓
Planner Agent
↓
Evidence Agent
↓
QA Agent
↓
Report Agent
↓
Output
```

All agents operate on the same shared state.

Why Functional Agents

Advantages:

```
No domain ownership conflicts
Multi-domain research supported
Shared profiles
Shared research state
Easier team scaling
```

Example:

```
{
  "profiles": [
    "smr",
    "ai_data_centers"
  ]
}
```

The same workflow can analyze both domains simultaneously.

Current Architecture

```
Research Object
↓
Profile(s)
↓
Functional Agents
↓
Evidence
↓
Findings
↓
Reports
```

Core platform:

```
research_agent
```

New orchestration layer:

```
functional_agents
```

Shared configuration:

```
profiles
```

Capability Matrix

Capability	Status
Local Retrieval	✓
Web Retrieval	✓
Evidence Extraction	✓
Evidence Ranking	✓
Contradiction Detection	✓
Research Gap Detection	✓
Coverage Analysis	✓
Evaluation	✓
Regression Testing	✓
Semantic Benchmarking	✓
Benchmark Hygiene	✓
Research Objects	✓
Profile Routing	✓
Profile Editing UI	✓
Functional Agent Framework	✓
Team-Scale Agent Architecture	In Progress

Current Project State

The platform has evolved from:

Question

↓

Answer

into:

Research Platform

with:

Durable Research State
Profile-Based Knowledge
Functional Agent Workflows
Objective Quality Measurement
Regression Protection

The next stage of development is focused on:

Functional Agent Intelligence

rather than infrastructure construction.

The platform foundation is now in place.